

October 2026

Watching the Deployment, Not the Call: Continuous Performance and Drift Monitoring for Deployed Clinical Agents

Jim Schwoebel[1,*] QUOKKAGUARD Contributors[1]

[1] Quome

Abstract

A clinical agent that is safe at release does not stay safe. Practice patterns shift, populations change, an upstream model is swapped beneath it, or an attacker reshapes its behavior gradually to stay under per-call detectors. The prior layers of this program decide *each call* — they firewall prompts [11], attest the harness [8], make the action log non-repudiable [9], authorize tools in context [7], control data egress [6], and bind agent identity [10] — but *nothing watches the deployment over time*. We build `qfire monitor`, an off-hot-path layer that consumes the signed decision stream and runs sequential, memory-bounded detectors for three deployment failure modes: performance drift, autonomy creep, and behavioral anomaly. We release **HAARF-Drift**, a deterministic, fully synthetic benchmark of 168 temporal agent-decision streams (1000 cases each) with labeled, injected drift/creep/anomaly episodes of known onset and magnitude. **Headline:** across four drift detectors, *no single detector dominates* — CUSUM detects an injected degradation within 1.0–8.1 cases (large→small magnitude, ROC AUC 1.00), while ADWIN is robust but conservative on small drift (AUC 0.83). Autonomy-creep monitoring reaches 1.00 recall at 0.00 false-alarm rate; an *unsupervised* behavioral monitor reaches 1.00 recall at 0.00 false-positive rate *with no labels* — the regime where outcome-based monitoring degrades (detection falls to 0.56 and latency rises to 364 cases under delayed/partial labels). All detectors run at sub-microsecond to 0.22 µs per event with bounded memory. The central result: deployment-time safety needs *several complementary signals*, including a label-free one, because the failure that hides from outcomes is exactly the one an adversary aims for. All code, data, and result artifacts are released; all data are synthetic.

Keywords: drift detection; post-market surveillance; autonomy monitoring; concept drift; conformal martingales; clinical AI safety; total product lifecycle.

Framework: implements HAARF control C1.4/C1.5 [12] — the Healthcare AI Agents Regulatory Framework, the source of the QUOKKAGUARD program.
Code & data: `github.com/quome-cloud/qudrift`
Correspondence: jim@quome.com

[*] Corresponding author.

1 Introduction

A deployed clinical agent is approved once and then runs for months. Over that window the world moves: case mix changes with the season, a vendor silently updates the underlying model, a guideline is revised, or an attacker who has compromised the agent changes its behavior *slowly* so that no single call trips a detector. The prior layers of the QUOKKAGUARD program are all *point-in-time* controls. QFIRE firewalls the prompt [11]; the pristine-weights work attests the harness rather than just the model [8]; the tamper-evident log makes the action record non-repudiable [9];

the policy layer authorizes tools in context [7]; the egress layer controls what data leaves a call [6]; the identity layer binds agents in a multi-agent setting [10]. Each of these answers "is *this call* safe?". None answers "is the *deployment* still safe?". That second question is the deployment half of the total-product-lifecycle that regulators now expect for adaptive medical software [14], and it is the question dataset shift makes unavoidable at the bedside [2].

Why batch evaluation is not enough. The obvious answer — re-evaluate the agent periodically against a labeled test set — fails in deployment for two reasons. First, labels are *late*: a clinical outcome that would confirm a bad decision may not arrive for days or weeks, and many decisions never get a clean label at all. Second, the most dangerous degradation is the one that does *not* yet show up in outcomes — covariate drift or a quiet behavioral shift that precedes harm. A monitor must therefore run *sequentially* on cheap proxy signals, raise an alarm within a bounded number of cases, control its own false-alarm rate, and keep working when labels are scarce. This is the classical concept-drift setting [3], here under the constraints of a clinical gateway and an adversary.

What we build. We add `qfire monitor`, a new module in the QUOKKAGUARD gateway. It is an *off-hot-path* consumer of the signed decision stream produced by the audit layer [9]: the detectors never touch the request path, so the layer adds *zero* per-call latency. It watches three deployment failure modes (§2): performance drift (D1), autonomy creep (D2), and behavioral anomaly (D3), all under the silent-failure modifier of delayed/partial labels (D4). It raises *graduated* alerts — log, notify, throttle, halt — and exports real-world evidence for lifecycle re-assessment.

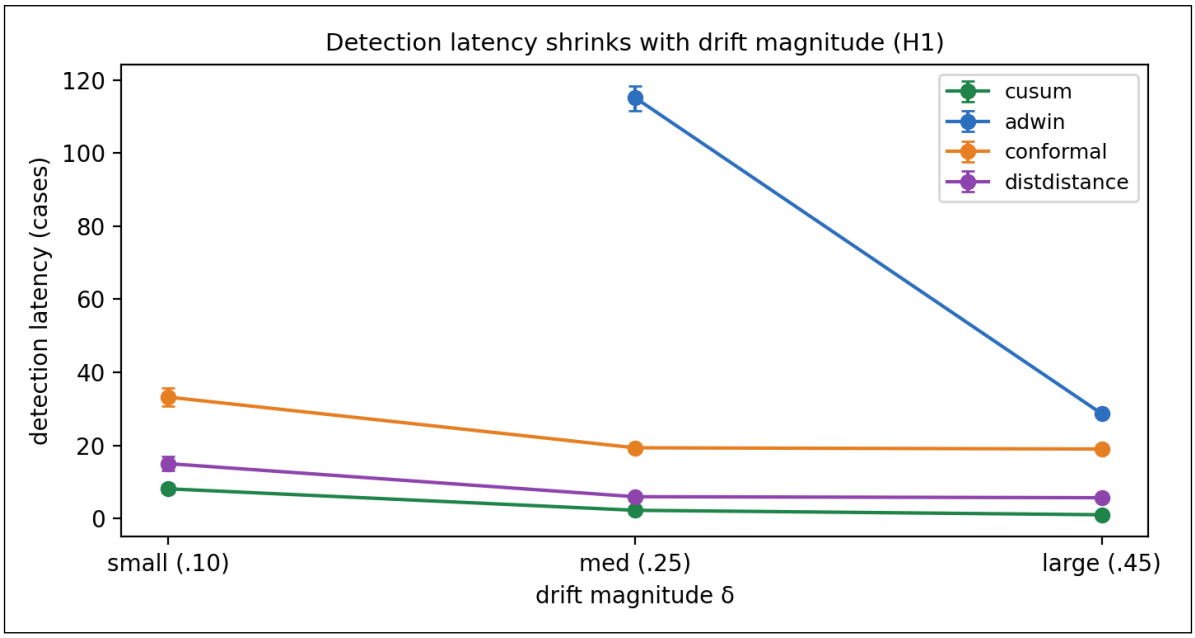


Figure 1: **The paper in one figure.** Detection latency (cases from the true onset to the first alarm; bootstrap 95% CI) versus injected drift magnitude δ, for four sequential detectors on HAARF-Drift. CUSUM and the distributional-distance monitor catch even small drift in a handful of cases; the conformal martingale is steady; ADWIN is *robust but conservative* — it misses the smallest drift entirely (no point at δ=0.10) and pays a long latency at medium δ. No single detector dominates, which is the design argument: a deployment monitor should run *several* complementary detectors rather than bet on one. Every detector here works from a cheap proxy signal and needs no labels.

A review of related work reveals three families (Table 1): periodic batch re-evaluation, streaming concept-drift detectors from the ML-systems literature, and emerging AI-autonomy/post-market frameworks. The recurring gap is that none is wired *into a clinical agent gateway as an always-on, adversary-aware, label-robust layer*.

Contributions.

1. **The central result.** On HAARF-Drift, no single drift detector dominates, and outcome-based monitoring *degrades under delayed/partial labels* (detection 0.72→0.56, latency 60→364 cases), while a *label-free* behavioral monitor holds at 1.00 recall, 0.00 FPR — so deployment-time safety requires several complementary signals, including an unsupervised one.
2. **The system.** `qfire monitor`: four sequential, memory-bounded drift detectors (CUSUM, ADWIN, conformal-martingale, distributional-distance), an autonomy meter against an authorized envelope, an unsupervised per-agent behavioral baseline, a graduated alert ladder, and a real-world-evidence exporter — all off the hot path.
3. **The benchmark. HAARF-Drift**: 168 deterministic, fully synthetic temporal streams (1000 cases each) with labeled, injected drift, autonomy-creep, and behavioral-anomaly episodes of known onset and magnitude, released with a one-command regenerator.
4. **Deployment evidence.** The whole monitor runs at sub-µs to 0.22 µs per event with bounded memory (§5.5); it is an off-hot-path read-side over an existing signed log, so it imposes no per-call cost.

Positioning. The closest prior line is the streaming concept-drift literature [5, 1, 3, 15]. We do not invent a new detector; we ask which of these classical detectors actually serve a *clinical agent gateway* under late labels and adversarial drift, and we wire the answer in as an always-on layer that feeds the program's oversight [7] and lifecycle machinery.

2 Problem Setting

We monitor a stream of agent decisions. The deployment failure modes are:

- **D1 Performance drift.** Outcome quality declines over time — covariate shift, label shift, or a silent upstream-model update.
- **D2 Autonomy creep.** The agent acts autonomously more often, or on higher-risk actions, than its authorized envelope permits.
- **D3 Behavioral anomaly.** A sudden shift in tool-use or decision patterns (compromise, misconfiguration) with no labeled outcome to confirm it.
- **D4 Silent failure.** Degradation is invisible because labeled outcomes arrive late or not at all. D4 is a *modifier* on D1–D3, not a separate stream — it is the regime that defeats batch evaluation.

Signals. The monitor reads a `MonitorEvent` per decision. In deployment this is *joined* from a signed audit record [9] — which already carries a timestamp, the terminal verdict, the maximum detector score, and per-node latency — plus three fields fed back out of band: a possibly-lagged clinical outcome (D1/D4), the autonomy level/risk tier of the action taken (D2), and the agent identity and tool invoked (D3). The benchmark emits the joined event directly so the detectors are exercised exactly as on the deployed read-side path.

Approach	What it does	Gap this paper targets
Periodic batch re-evaluation [2]	re-score against a labeled set on a schedule	late labels; misses pre-outcome drift; not continuous
Streaming drift detectors [5, 1, 15]	sequential change detection on a data stream	not wired to an agent gateway; no autonomy/behavioral/adversarial framing
AI lifecycle / SaMD frameworks [14, 12]	require post-market monitoring	specify *that*, not an always-on *mechanism*
This work	off-hot-path, multi-signal, label-robust monitor in the gateway	—

Table 1: Positioning against representative approaches to keeping a deployed model safe.

3 System Design

3.1 Overview and components

`qfire monitor` (Rust, 16 unit tests) is a read-side consumer of the signed decision stream. It comprises: **four drift detectors** over a scalar proxy signal, an **autonomy meter**, an **unsupervised behavioral baseline**, a **graduated alert ladder**, and a **real-world-evidence (RWE) exporter**. Every drift detector implements one sequential interface — `observe(case, x)` returning an optional `DriftSignal` — and is memory-bounded (CUSUM $O(1)$; ADWIN $O(\log n)$ buckets; conformal/distance $O(\text{window})$). This is the deployability claim: the monitor is a constant- or log-space pass over a stream.

Drift detectors. (1) **CUSUM** [5] accumulates two-sided cumulative sums of the proxy against an in-control mean with slack k and threshold h. (2) **ADWIN** [1] maintains an exponential histogram and shrinks an adaptive window whenever a Hoeffding-bounded mean difference is significant. (3) The **conformal martingale** [15, 16] bets on conformal p-values; by Ville's inequality the betting martingale bounds the false-alarm rate by α under exchangeability [13]. (4) **Distributional distance** compares a fixed reference histogram to a sliding window via the population stability index or a Kolmogorov–Smirnov statistic. They embody four different assumptions about *what changed*, which is why they are complementary (§5).

Autonomy meter. An `AutonomyEnvelope` (maximum autonomous risk tier, maximum autonomous fraction over a window) models the surface the forthcoming regulatory passport will supply. The meter flags an action that is autonomous above the authorized tier immediately, and flags sustained autonomous-fraction creep over the window.

Behavioral baseline. For each agent, a categorical tool-use profile is learned over a warmup window; a sliding window is then scored against it by population-stability divergence (a previously-unseen tool is a strong signal). This path uses *no labels* and is the answer to D4.

Graduated alert ladder. Detector severity maps to `log` → `notify` → `throttle` → `halt`. The ladder tallies per-level volume so that only the clinician-facing levels (notify/throttle/halt) count against an alert-fatigue budget — the seam to the program's oversight router.

The monitor as an off-hot-path consumer

```
signed decision stream (quledger)  -->  MonitorEvent (+ outcome, autonomy, tool)
  drift:     cusum | adwin | conformal | distdistance   (proxy signal, no labels)
  autonomy:  meter vs authorized envelope               (risk tier + fraction)
  behavior:  per-agent baseline                         (unsupervised, D4)
  ladder:    log -> notify -> throttle -> halt          (clinician budget)
  rwe:       export evidence -> passport / equity reporter
```

Figure 2: The monitor reads the signed decision stream off the hot path and runs each signal independently, so it adds no per-call latency and degrades gracefully if a feed is late.

3.2 Implementation

The module is `src/monitor/` in the gateway crate; a `qfire monitor --stream` subcommand replays a decision stream through the detectors and prints the alert ladder plus an RWE summary. All experiments run offline over HAARF-Drift via a `driftbench` binary. Everything is deterministic given the seed.

4 HAARF-Drift

Existing drift benchmarks [4] target image/tabular distribution shift, not *temporal agent-decision streams with autonomy and behavioral structure*, and contain no labeled adversarial creep. We therefore build **HAARF-Drift**: 168 deterministic, fully synthetic streams (1000 cases each; seed 42), released with a one-command regenerator.

- **Drift** (54 onset + 18 clean): an outcome-failure proxy in $[0, 1]$, stationary then a step of known magnitude $\delta \in \{0.10, 0.25, 0.45\}$ at a labeled onset t_0, across three families — covariate (variance inflation), label (mean step), and model-update (step) shift.
- **Autonomy** (24 creep + 24 clean): per-case autonomous flag and risk tier; creep raises the autonomous fraction and tier past the envelope at t_0.
- **Behavior** (24 anomaly + 24 clean): per-case tool; an anomaly injects an abrupt tool-distribution shift (a previously-unseen tool) at t_0.

A sidecar records, per stream, the onset index, magnitude, and type. All data are synthetic; no protected health information is used.

5 Experiments and Results

We measure detection latency (cases from t_0 to the first alarm at or after onset; an alarm before t_0 or on a clean stream is a false alarm). Rates carry Wilson 95% CIs [17]; latencies carry bootstrap 95% CIs; ROC/AUC sweeps each detector's threshold.

5.1 E1 — drift detection and the no-dominator result

At their operating points all detectors except ADWIN detect every onset (54/54, 54/54, 54/54); ADWIN detects 36/54 because its conservative Hoeffding bound misses the smallest magnitude. Detection latency shrinks with magnitude (Figure 1): CUSUM falls from 8.1 cases at δ=0.10 to 1.0 at δ=0.45; the distance monitor from 15.0 to 5.7; the conformal martingale is steady around 19.0; ADWIN pays 115.2 cases at medium δ. The ROC (Figure 3) confirms the picture: AUC 1.00 (CUSUM), 1.00 (conformal), 1.00 (distance), and 0.83 (ADWIN). *No single detector dominates*: CUSUM is best for mean shifts, the distance monitor is fast and catches variance-only (covariate) drift that a mean detector cannot, the conformal martingale gives a distribution-free false-alarm guarantee, and ADWIN is the most conservative. This is why the monitor runs all four.

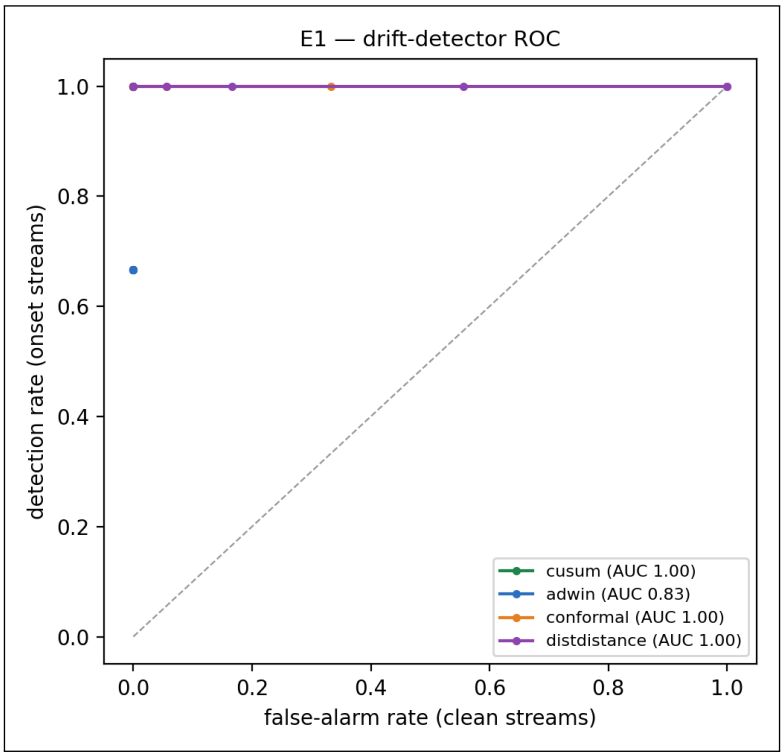


Figure 3: **E1 ROC.** Detection rate over onset streams versus false-alarm rate over clean streams, sweeping each detector's threshold. CUSUM, conformal, and distance reach the top-left corner (AUC 1.00); ADWIN tops out at AUC 0.83 because it cannot catch the smallest drift at any threshold without alarming on clean streams.

5.2 E2 — delayed and partial labels (D4)

We feed the *outcome*-based signal to CUSUM and vary label delay and label fraction (Figure 4). With full, immediate labels, outcome-CUSUM detects 0.72 of onsets at latency 60 cases. Delay

adds directly to latency (210 cases at a 150-case delay) and sparsity both raises latency and lowers recall (0.56 at a 20% label fraction; up to 364 cases combined), all at a false-alarm rate ≤ 0.056. Crucially, recall *caps at 0.72* even with perfect labels, because covariate-family drift inflates variance without yet moving the mean outcome — a proxy-validity failure that outcome monitoring cannot escape. This is the quantitative case for not relying on labeled outcomes alone.

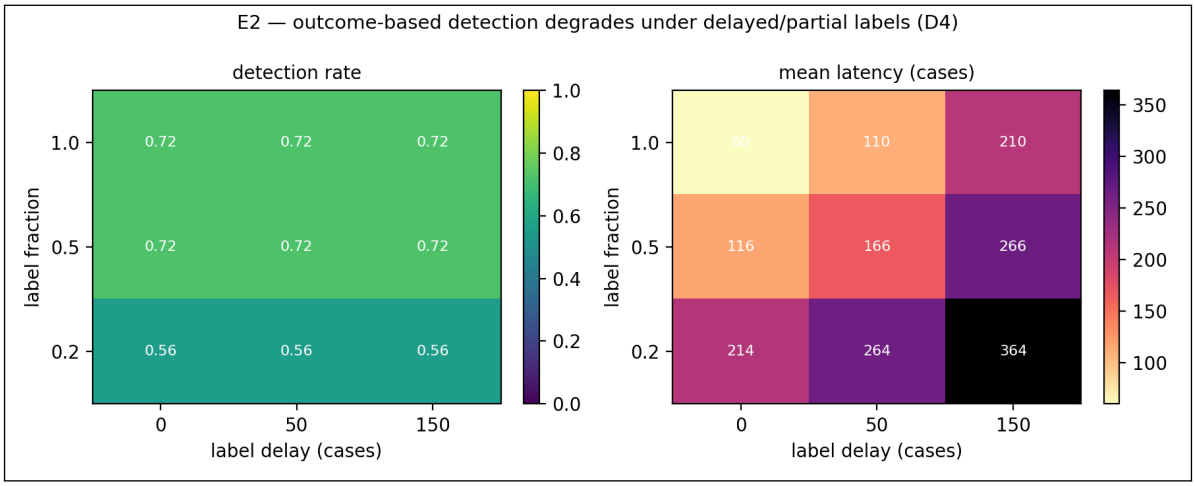


Figure 4: **E2 — outcome-based detection degrades under D4.** Detection rate (left) and mean latency (right) as label delay (columns) and label fraction (rows) vary. Latency grows with both; recall falls with sparsity and is capped by covariate drift that outcomes do not reflect — motivating the label-free behavioral path (E4).

5.3 E3 — autonomy creep

Against authorized envelopes spanning maximum autonomous fractions 0.4–0.7, the autonomy meter catches creep at 1.00 recall and 0.00 false-alarm rate, with a latency of 0.2 cases past onset (Figure 5) — because a single autonomous action above the authorized tier is flagged immediately. This is the clean case: autonomy is observable directly, without labels and without ambiguity, so it should be monitored directly rather than inferred from outcomes.

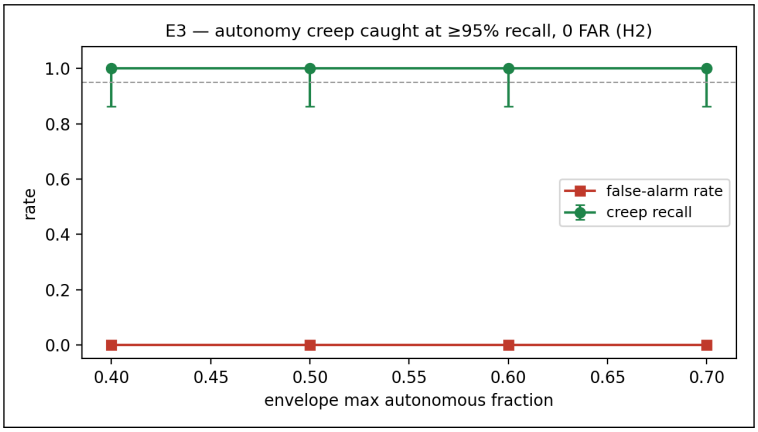


Figure 5: **E3 — autonomy creep.** Creep recall (Wilson 95% CI) and false-alarm rate versus envelope tightness. Recall is 1.00 at 0.00 false alarms across the swept envelopes (dashed line: 95% target).

5.4 E4 — unsupervised behavioral anomaly

The behavioral baseline uses no labels. Sweeping its threshold (Figure 6), it reaches 1.00 recall throughout; at the operating threshold 0.5 the false-positive rate is 0.00 (zero for any threshold ≥ 0.35) at a latency of 14.8 cases. Only an over-sensitive threshold (0.2) inflates the FPR (to ≈ 0.42). Where outcome monitoring stalls under D4 (E2), this label-free signal holds — the two paths are complementary, which is the structural argument of the paper.

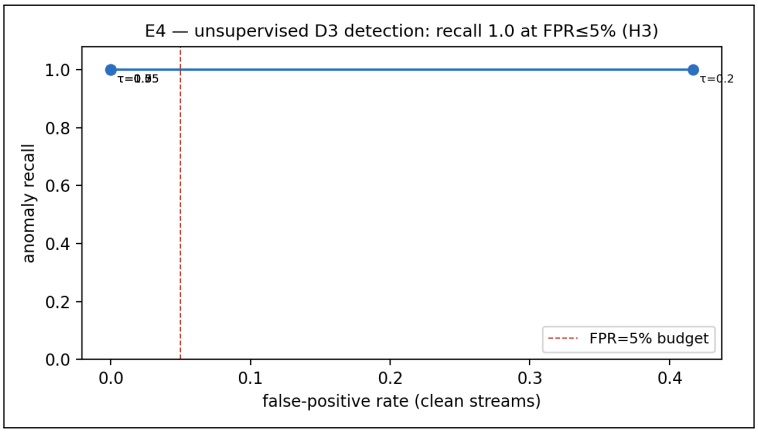


Figure 6: **E4 — unsupervised behavioral detection.** Anomaly recall versus false-positive rate as the threshold varies; the operating point sits at recall 1.00, FPR 0.00, well inside the 5% budget (dashed line). No labels are used.

5.5 E5 — monitoring overhead

Over 72000 events, per-event cost is 0.007 µs (CUSUM), 0.034 µs (conformal), 0.19 µs (ADWIN), and 0.22 µs (distance) (Figure 7); ADWIN's resident state stays bounded at 53 buckets. Because the monitor consumes the signed log off the hot path, this cost is never on the request critical path: the deployment imposes *no* per-call latency.

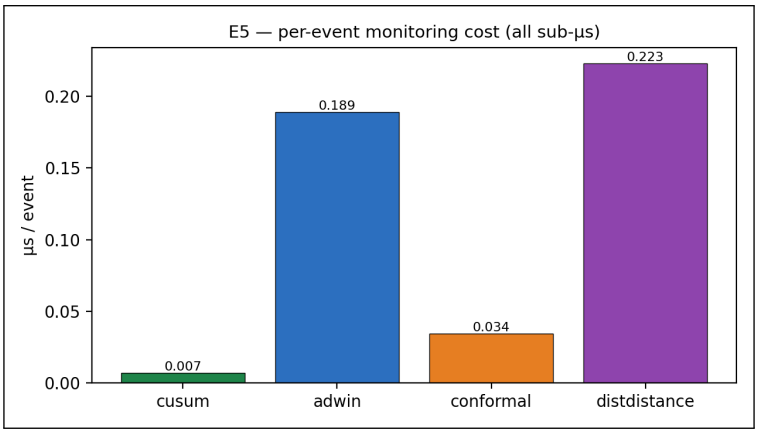


Figure 7: **E5 — overhead.** Per-event monitoring cost for each detector; all are sub-microsecond to a fifth of a microsecond, with bounded memory.

6 Discussion and Limitations

Interpretation. The results say something simple and a little uncomfortable: there is no one detector to deploy. Mean-shift, variance-shift, exchangeability, and distributional detectors each catch what the others miss, and the signal that survives the silent-failure regime (D4) is the *unsupervised* behavioral one — precisely because an adversary degrading an agent will avoid moving the labeled outcome for as long as possible. A deployment monitor is therefore a small *ensemble* of cheap detectors plus a label-free baseline, not a single statistic.

Relationship to HAARF. The layer realizes continuous performance monitoring (C1.4) and real-world evidence (C1.5) via the drift detectors and RWE exporter, autonomy performance monitoring (C6.6) via the autonomy meter, and behavioral anomaly detection (C5.5) via the unsupervised baseline [12]. The graduated ladder is the seam to the oversight router, and the RWE exporter is the seam to the regulatory passport and equity reporter.

Limitations. HAARF-Drift is synthetic: it lets us control onset and magnitude exactly and reason about detection latency, but real clinical streams will have richer, confounded structure, and the proxy-signal-validity threat (E2's ceiling) is more severe in practice. The authorized envelope and the alert router are modeled locally; wiring them to the real passport and oversight layers is the subject of the companion papers. We report detection latency on injected step/episode drift; gradual ramps and recurring seasonal drift are future work. Finally, the adversary here is a behavioral one — an attacker who can also forge the monitored feed is out of scope and is the province of the signed-audit layer [9].

7 Conclusion

A clinical agent that passes review can still fail in the field, and the failures that matter most are the ones that hide from labeled outcomes. `qfire monitor` watches the deployment rather than the call: an off-hot-path ensemble of sequential, memory-bounded detectors for drift, autonomy creep, and behavioral anomaly, plus an unsupervised path for the silent-failure regime, at submicrosecond per-event cost. The next steps consume its output: the regulatory passport ingests its real-world evidence to gate change, and the oversight router consumes its graduated alerts to route only genuinely ambiguous cases to clinicians.

Declaration of generative AI use

During the preparation of this work the authors used *Anthropic Claude (Claude Code)* for drafting boilerplate, generating and debugging the Rust monitor module, the synthetic data generator, and the figure-plotting code, and for copy-editing. After using this tool, the authors reviewed and edited the content and take full responsibility for the publication. No generative-AI system was used to generate or fabricate data, interpret results, or produce clinical conclusions, and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the HAARF-Drift generator, the `driftbench` harness, and the analysis scripts are available at `github.com/quome-cloud/qudrift`. All patient data are synthetic; no real protected health

information was used.

Author contributions

J.S. and the QUOKKAGUARD contributors conceived the study, designed and implemented the monitor layer and benchmark, ran the experiments, and wrote the manuscript; all authors reviewed and approved it.

Competing interests

The authors declare no competing interests.

Funding

This work received no specific external funding.

A Reproducibility

From a checkout of the repository:

```
python3 scripts/011-drift-monitoring/gen_drift.py        # HAARF-Drift (seed 42)
cargo run --release --bin driftbench -- --exp all        # E1-E5 -> results/011-*
python3 scripts/011-drift-monitoring/summarize.py        # CIs, ROC/AUC, figures
cd papers/011-drift-monitoring && make paper             # this PDF
```

Every figure and number regenerates from the seeded run; detection metrics are deterministic, and per-event timings (E5) are the only run-dependent quantities.

References

[1] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proc. SIAM Int. Conf. on Data Mining (SDM)*, pages 443–448, 2007.

[2] Samuel G. Finlayson, Adarsh Subbaswamy, Karandeep Singh, et al. The clinician and dataset shift in artificial intelligence. *New England Journal of Medicine*, 385:283–286, 2021.

[3] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.

[4] Pang Wei Koh et al. Wilds: A benchmark of in-the-wild distribution shifts. *Proc. ICML*, 2021.

[5] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[6] James Schwoebel. Phi egress control and covert-channel defense for clinical ai agents, 2026. Quome, QUOKKAGUARD program. https://github.com/quome-cloud/qudam.

[7] James Schwoebel. Policy-as-code authorization for clinical ai agents: Context-aware decisions and a cedar-vs-rego comparison, 2026. Quome, QUOKKAGUARD program. https://github.com/quome-cloud/quwarden.

[8] James Schwoebel. Pristine weights, poisoned goals: Intent-mutation attacks on self-improving clinical agents and attestation-rooted harness defenses, 2026. Quome, QUOKKAGUARD program. https://github.com/quome-cloud/quanchor.

[9] James Schwoebel. Tamper-evident action provenance for confidential clinical ai agents, 2026. Quome, QUOKKAGUARD program. https://github.com/quome-cloud/quledger.

[10] James Schwoebel. Who is calling? cryptographic agent identity for malicious agents in multi-agent clinical systems, 2026. Quome, QUOKKAGUARD program. https://github.com/quome-cloud/quorum.

[11] James Schwoebel, Ingrida Semenec, Jenia Rousseva, Martin G. Frasch, Rome Thorstenson, and Manish Bhatt. Beyond injection detection: A positive-security prompt firewall that closes the scope and PHI gap SOTA classifiers miss in healthcare. *medRxiv*, 2026. Preprint, https://doi.org/10.64898/2026.06.04.26354950.

[12] Jim Schwoebel, Martin Frasch, Art Spalding, Ed Sewell, Phil Englert, Ben Halpert, Collin Overbay, Ingrida Semenec, and Joel Shor. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. medRxiv preprint, https://doi.org/10.64898/2026.04.09.26350519, April 2026.

[13] Glenn Shafer and Vladimir Vovk. *Game-Theoretic Foundations for Probability and Finance*. Wiley, 2019.

[14] U.S. Food and Drug Administration. Artificial intelligence/machine learning-based software as a medical device (samd) action plan. *FDA*, 2021.

[15] Denis Volkhonskiy, Evgeny Burnaev, Ilia Nouretdinov, Alexander Gammerman, and Vladimir Vovk. Inductive conformal martingales for change-point detection. In *Conformal and Probabilistic Prediction and Applications (COPA)*, 2017.

[16] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

[17] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.